推荐经典模型和 Loss 推演 Part 2

从历史状态、兴趣分工到排序梯度：完整往返推演

Bo Hou

2026-10-30

本文接续 Part 1，整理用户历史编码、多兴趣召回、MoE 路由均衡，以及 pointwise、pairwise、listwise loss。重点保留讨论中的追问、反驳和修正：不是把模型名称排成清单，而是把“为什么这样设计”讲清楚。

文中对话为编辑后的重述，不是逐字聊天记录。原论文定义、我们构造的教学版本和讨论中的可选扩展会明确区分。本文是技术学习笔记，不主张研究原创性，也不把理论解释当成已经验证的工业结论。

阅读入口： Part 1 中文 PDF；Part 1 English PDF。

目录

1. 从 Part 1 接到哪里：四个独立问题
2. 历史编码的起点：我已经有 self-attention 了
3. 自己设计一个 GRU：写什么，写多少
4. 状态有没有丢信息，门控有没有学到预想语义
5. DIEN：让中间状态有明确的监督任务
6. 时间窗口怎么切，什么才是主任务目标
7. SASRec：把熟悉的 causal encoder 对齐到原始训练目标
8. 多兴趣：把一个用户向量变成 K 个
9. 从 hard selection 推到软分配与塌缩监控
10. 向量多样性和路由均衡不是同一个问题
11. 借鉴 MoE：单样本稀疏，整体避免闲置
12. Switch 均衡损失：两个比例为什么相乘
13. MIND 原论文实际做了什么
14. 海量数据能否自然解决兴趣闲置
15. 用户模型总表与时间线
16. 统一实验能说明什么，不能说明什么
17. Loss 的分界：训练单位，不是输出有没有标量
18. Pairwise logistic：从三元组到两边的梯度
19. Hinge、BPR 与 NDCG 加权梯度
20. Listwise：列表概率，而不只是模拟 NDCG 梯度
21. 模型、数据、loss、优化的实验怎么拆
22. 面试与 ML coding：把推演落到一个能跑的闭环
23. 纠错记录、接续问题与参考文献

1. 从 Part 1 接到哪里：四个独立问题

Part 1 已经推过双塔、用户历史、FM、DeepFM、Wide & Deep、DCN v1/v2、DLRM 和 DIN。继续讨论前，我重新修正了理解模型的框架。

最初说的是“数据、模型、优化”。后来发现 loss 应单独列出来：

问题	它决定什么	不能被什么替代
数据	样本、可用信息、监督与观察分布	一个新结构不能凭空补足没观察到的信号
模型结构	可表达的函数及哪些关系更容易表达	表达能力不等于优化目标
Loss	什么结果值得奖励、什么错误值得惩罚	优化器不会自动知道业务目标
优化	如何更新参数，是否稳定、高效地找到好解	loss 合理也不代表一定容易训练

评估和部署负责闭环：结果是否可靠、有实际价值，以及线上能否计算、更新和监测。

我的总结： 同一个打分网络可以采用 BCE，也可以采用 pairwise loss。模型不变，训练目标已经变了；优化器甚至可以完全不变。因此 loss 不该只是“优化方法”下面的一小项。

2. 历史编码的起点：我已经有 self-attention 了

讨论从 DIN 的历史加权继续。若不加位置或时间，同样的历史集合面对同一个候选，聚合结果不会因为事件顺序改变而变化。

举例：

- 用户 A：早期看喜剧，最近连续看惊悚。
- 用户 B：早期看惊悚，最近连续看喜剧。

两人看过的集合相同，当前兴趣却可能不同。

引导： 怎样让历史表示包含此前发生过什么？

我的反问： 这不是 self-attention 的东西吗？最后一个有效 token 的 embedding 已经能包含前面的历史。

这个反问是对的。我们在 Part 1 已经推过 causal self-attention，没有必要为了引出另一个论文名字重新假装这个问题尚未解决。

因此接下来的重点应变成两件事：

1. GRU 是另一种历史编码结构，基本递推如何理解？
2. 无论用 GRU 还是 attention，怎样让中间状态得到更直接的监督？

注意：causal mask 规定可见范围，位置或时间输入提供顺序线索；不能把“做了 attention”自动等同为“已经知道时间顺序”。

3. 自己设计一个 GRU：写什么，写多少

3.1 先把最小递推讲清楚

$$h_t = \text{GRU}(x_t, h_{t-1}).$$

x_t 是当前行为表示，h_{t-1} 是之前历史的状态，h_t 是读入当前行为后的新状态。

我的理解： 给定输入，通过网络得到输出，再把过去信息和最新输入合起来继续计算。

修正： 传给下一步的不是上一步原始输入，而是上一步隐藏状态。可以把它看成“之前历史的总结”。GRU 的参数跨时间共享，变化的是输入和状态。

它不是每一步训练一套新网络，也不是把前一步优化器更新后的参数当成“记忆”。递推状态和模型参数是两类东西。

3.2 我的门控设计直觉

我提出：把 $[h_{t-1}, x_t]$ concat 后，通过一个门控网络决定旧信息保留多少；再通过另一个变换生成更新内容，最后合成 h_t。

先固定本文符号：

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z),$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t.$$

本文中 z_t 表示**新内容写入比例**，$1 - z_t$ 表示旧内容保留比例。有些文献或框架反过来命名，两种约定都存在；看完整公式，不要只背 update gate 的名字。

z_t 通常是与状态同维的向量，逐坐标决定更新比例，不是所有信息只共用一个标量。

3.3 “Sigma 是 ReLU 吗？超过阈值才激活？”

这里 σ 特指 sigmoid：

$$\sigma(a) = \frac{1}{1 + e^{-a}} \in (0, 1).$$

它是平滑的比例门，不是超过某个阈值才开关。

- $z = 0.9$：这一维更偏向新候选内容。
- $z = 0.1$：这一维主要保留旧状态。

ReLU 无上限，不能直接解释为这里的凸组合比例。sigmoid 很大或很小时也可能饱和，门控并不意味着梯度问题自动消失。

3.4 候选新状态从哪里来？

最简教学版本是：

$$\tilde{h}_t = \tanh(W_h[h_{t-1}, x_t] + b_h).$$

两个网络职责不同：

- $\tilde{h}_t$：准备写什么。
- z_t：准备写多少。

tanh 是双曲正切，把数值压到 $(-1, 1)$。它的英文名称里没有一个额外的 H 参数。

我的猜测： 接近正一或负一的信息可能更重要？

修正： 数值绝对值大不等于语义更重要，更不能从激活值直接读出“该记住什么”。后续预测如何使用这个坐标，才决定它的任务价值。

3.5 再引出 reset gate

生成新候选内容时，旧历史未必需要完整参与。因此加入：

$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r),$$

$$\tilde{h}_t = \tanh\left(W_h[r_t \odot h_{t-1}, x_t] + b_h\right).$$

这是一种常见的 GRU 表达；不同实现对 reset 的乘法位置和 bias 处理可以有差别。

reset gate 管“构造新内容时参考多少旧状态”，update gate 管“新旧内容最终如何混合”。这比死记多个门的名字更容易对上计算图。

4. 状态有没有丢信息，门控有没有学到预想语义

4.1 我最初的犹豫

我的问题： GRU 看起来有点 attention 的意思，但固定状态会不会丢信息？LSTM、RNN 不是也一直有这个问题吗？

需要区分两类困难：

1. **表示压缩：** 越来越多的历史不断写进一个有限维状态，细节可能被压掉。
2. **优化路径：** 梯度沿很多递推步骤传播，可能衰减或放大。

门控和残差式更新能改善信息路径，却不保证所有有用历史都被保留。

Attention 的一个优势是可以保留不同位置的表示，在预测时直接访问相关位置，不必只依赖最后一个递推总结。早期 encoder-decoder attention 的动机之一就是固定长度压缩瓶颈。

但 attention 同样有上下文长度、有限维度、聚合和学习目标的限制。它不是无损记忆装置。

4.2 结构明确，不代表语义自动成立

我认为 FM/DCN 的结构更容易和预期联系起来：明确做内积、乘积交叉。而 GRU 中称某个变量为“兴趣状态”，不等于它就真的表示了想要的兴趣。

这个区分很重要：

- FM 明确计算交互，DCN 明确构造输入坐标乘积。
- GRU 明确执行选择性状态更新。
- Self-attention 明确执行学习到的匹配与加权。

以上是计算事实。至于“内积就是语义”“某个门负责长期兴趣”“某维是喜剧”，则是需要证据的解释。

乘积也不等于统计相关系数；QK 内积也不保证学到人类直觉中的语义相似。

4.3 为什么要辅助监督

我的推论： 这种循环网络展开后其实很深。可能需要给中间步骤加 cost function，帮助它学到想保留的状态。

对，但辅助 loss 通常监督中间状态的**可预测行为**，不是直接告诉某个 gate “你必须成为长期记忆门”。

可以把“状态应该表示兴趣”落成一个操作性目标：用它预测下一次真实交互。这提供更明确的训练压力，不构成内部语义的数学保证。

5. DIEN：让中间状态有明确的监督任务

5.1 重新提出真正的问题

若最终只有一次点击 label，中间状态主要通过长链路间接获得信号。能否让每一步 h_t 也承担一个预测任务?

训练记录可构造成：

$$(h_t, v_{t+1}^{+}, v_{t+1}^{-}),$$

其中 v_{t+1}^{+} 是下一次实际点击的 item，v_{t+1}^{-} 是按明确协议采出的负例。

我的回答： 那当然就是让历史状态和真实下一次点击更接近，与采样负例更远。可以内积或 cosine，再做 softmax CE。

这个方案成立。关键是把它与原版 DIEN 的具体目标区别开。

5.2 教学版本与原论文对齐

我们可设计一个共享二分类头：

$$g_\psi(h_t, v) = \mathrm{MLP}_\psi([h_t, v]), \qquad D_\psi(h_t, v) = \sigma(g_\psi(h_t, v)).$$

每步的辅助损失：

$$\ell_t^{\mathrm{aux}} = -\log D_\psi(h_t, v_{t+1}^{+}) - \log\left(1 - D_\psi(h_t, v_{t+1}^{-})\right).$$

这是下一行为区分任务：让状态携带能区分真实后续行为和负例的信息。BCE 并不必然要求向量的欧氏距离变小；如果打分头是 MLP，“接近”应理解为任务匹配更强。

DIEN 原版使用逐步的正负下一行为二分类监督，并与主 CTR loss 联合训练。公开数据与工业数据的负例来源也不同，不能把所有负例都说成随机未点击物品。

5.3 padding 和权重必须有定义

设 m_{bt} 表示 batch 中用户 b 的时间位置 t 是否有效。一个按有效位置平均的教学目标：

$$L_{\text{aux}} = \frac{\sum_{b,t} m_{bt} \ell_{bt}^{\text{aux}}}{\sum_{b,t} m_{bt}}, \qquad L = L_{\text{CTR}} + \lambda_{\text{aux}} L_{\text{aux}}.$$

也可以先按用户平均再按 batch 平均，但那对长短历史的权重不同。不要把不同 reduction 当成没有意义的实现细节。

mask 不是把 padding 当作“不点击”负例。对没有下一个有效事件的位置，也不应构造伪监督。

5.4 完整 DIEN 还有候选相关兴趣演化

我们讨论的“GRU + 辅助 loss”只对齐了 interest extractor。完整 DIEN 还用候选相关 attention 调节后续兴趣演化；其 AUGRU 将 attention 强度用于调制更新门。它并非简单地取第一个 GRU 的最后状态就结束。

因此，GRU hidden 接一个双塔是可行设计，但不应直接称为原版 DIEN。原版候选相关模块更自然位于 ranking。

6. 时间窗口怎么切，什么才是主任务目标

我试图把它接回前面的训练管线：用户有 100 条历史，切长度 30 的窗口，内部上一步预测下一步，同时主任务预测窗口末尾。

必须先说明“30”包含不包含目标。

6.1 协议 A：30 个事件中，最后一个是目标

$$[a_1, \dots, a_{29}] \longrightarrow a_{30}.$$

- encoder 输入前 29 个事件，生成 $h_1, \dots, h_{29}$。
- 内部辅助任务可以用 h_t 预测 a_{t+1}，例如 $t = 1, \dots, 28$。
- 第 30 个真实交互可以作为主任务正例候选，另配主任务负例，或使用曝光日志中真实点击 label。

若也给 $h_{29} \to a_{30}$ 加辅助监督，这是额外且可能重复的监督，应明确计算权重。第 30 个事件无论如何不能先作为 encoder 输入，再让模型预测它。

6.2 协议 B：30 个事件全部是历史

$$[a_1, \dots, a_{30}] \longrightarrow a_{31}.$$

100 个事件按固定长度历史滑动，可得到 70 个这样的样本。协议 A 若枚举完整 30 事件片段，则可得到 71 个片段；这只是端点计数差别，不是模型差异。

训练集内部可以随机 batch。严格时间评估仍应先设全局切分，避免较晚训练窗口包含较早测试目标。用户状态、统计特征和预训练 encoder 的数据时间也应匹配评估协议。

7. SASRec：把熟悉的 causal encoder 对齐到原始训练目标

我们已经推过：物品 embedding 加位置信息，causal self-attention 编码前缀，得到每个位置状态 h_t，预测下一 item。

写一个简化打分：

$$s_{tj} = h_t^\top e_j.$$

单独一个 prefix 可以用来训练，也可以在一次序列前向中对多个有效位置监督。线上常用最后有效状态作为当前用户表示。

重要对齐：**原始 SASRec 使用带采样负例的二分类交叉熵，而不是因结构是 Transformer 就自动使用全量 item softmax。**后续实现可改 CE、采样和训练协议，结果也会变。

因此应分别记住：

- 结构：causal attention 历史 encoder。
- 目标：下一物品预测。
- 原版 loss：正例与采样负例的 BCE。
- 常见扩展：不同 softmax CE 与负采样协议。

不要把“结构相近”和“训练目标完全相同”混成一回事。

8. 多兴趣：把一个用户向量变成 K 个

8.1 最自然的输出改造

给定用户输入 X_u，把一个向量改成：

$$H_u = f_\theta(X_u) \in \mathbb{R}^{K \times d}.$$

这里是 K 个 d 维向量，不是“一个 K 维向量”。

它可以是历史 encoder 的多头输出，也可以由更简单的网络生成；它们都是中间激活值，梯度更新生成它们的参数。

8.2 “这不就是 lookup table？”

我最开始认为：生成 K 个候选，找匹配最大的索引，再让 CE 更新这个位置。

需要区分两种参数化：

1. user ID 直接查 $[K, d]$ 表，梯度更新被查出的参数行。
2. 用户历史、profile 等进入 encoder，输出 $[K, d]$，梯度经过输出回传到上游网络。

我们后面的讨论主要是第二种。不能一会儿说动态生成，一会儿又把每个中心当成独立存储的 user 参数。

8.3 “像 K-means 求中心”

这个直觉很好：历史行为是点，多种兴趣类似中心，行为按关联程度分到不同中心，再聚合。

但需要分清：

- 有的模型直接一次前向生成 K 个向量，不含显式聚类迭代。
- 有的模型确实反复执行软分配与中心聚合。
- 外层还有推荐 loss 的梯度更新参数。

一次前向里的 routing 迭代，和跨训练 step 的 optimizer 更新，是两个不同层面的“迭代”。

9. 从 hard selection 推到软分配与塌缩监控

9.1 正例该靠近谁?

问题： 一个点击视频出现时，应让所有 K 个兴趣都靠近它吗?

我的回答： 最相关的一个靠近，其他都远离。

修正： 第一部分合理，第二部分太强。未选中的兴趣不自动等于负例；它们也可能相关，不能只因不是赢家就全部推远。

一个教学 baseline：

$$s(u,v) = \max_k h_{u,k}^\top e_v.$$

再将该分数用于正负候选训练。hard max 对当前 pair 的梯度主要经过赢家，但不同候选可能选中不同兴趣。

9.2 什么叫塌缩?

我们区分了两个问题：

现象	表现	结果
使用失衡	几乎所有输入都由同一分支胜出	其他分支直接学习信号少
表示重复	K 个向量方向或打分行为高度相似	即使轮流胜出也没兴趣分工

某一个视频到几个中心的距离相近，不足以证明塌缩。更关键是不同候选下都几乎相同，或者长期只有一个分支获得作用。

9.3 初始化与富者愈富

我提议随机初始化，尽量让起点不同。

这能打破对称，但不保证持续分工。hard selection 下，初期略强的分支可能持续得到更多梯度；相同初始化也可能让输出一开始完全一样。

随机初始化通常不叫 Bayesian 初始化。只有引入明确先验、后验或相应推断过程时，Bayesian 才是有定义的说法。

9.4 温度控制的是分配，不是中心位置

用候选相关权重：

$$a_k(u,v) = \frac{\exp(h_{u,k}^\top e_v/\tau)}{\sum_j \exp(h_{u,j}^\top e_v/\tau)}.$$

- 温度低，权重集中。
- 温度高，权重分散。

这叫 soft assignment 或 soft routing，不是 soft label。label 没有因此变成概率标签。

可以用 $\bar{h} = \sum_k a_k h_k$ 再打分，或选其他平滑 max 的目标。仅写一个 softmax 而不说明它如何进入 loss，尚未定义完整算法。

软聚合让更多分支有梯度路径，但梯度可能仍很小，且可能让表示变得相似。它不能保证消除塌缩。

10. 向量多样性和路由均衡不是同一个问题

10.1 “把中心之间距离拉大”

我提议正则化：K 个中心之间越远越好。

如果不约束长度，模型可以仅放大范数来增加距离。为比较方向，可先做：

$$\hat{h}_k = \frac{h_k}{\|h_k\|_2 + \epsilon}.$$

它不是冻结表示，也不是逐维的标准化，而是去掉向量整体尺度的影响。

一种**可选教学正则**：

$$L_{\text{div}} = \frac{1}{K(K-1)} \sum_{k \neq j} (\hat{h}_k^\top \hat{h}_j)^2.$$

它鼓励方向近似正交，不是 MIND 原版目标。若 $K > d$，不可能所有向量两两正交；相关兴趣也未必应该强行正交。因此系数与形式都要用数据验证。

10.2 “归一化还是不能解决只有一个更新”

对。归一化限制的是尺度，方向正则处理的是重复，路由均衡处理的是使用分布。三个问题不同。

完全均衡的路由也可能对应一群相同向量；不同向量也可能长期不用。不要把一个指标当作所有健康性的替代。

11. 借鉴 MoE：单样本稀疏，整体避免闲置

我想到多兴趣分支和 MoE 很相似：gate 选几个 expert，是否也有训练监测与防过度集中的机制?

先区分两个层级：

- 单个输入通常只激活 top-k 个 expert，节省计算。
- 整个 batch 或长期流量避免只挤进少数 expert，使其他 expert 闲置。

不是每条输入都尽量均匀；那样会丢掉稀疏计算的目的。

对多任务 MoE，不同 task 使用 expert 的重叠程度可以监测，但重叠越少不必然越好。相关任务共享能力可能就是想要的结果。

可以监测选择比例、软权重总量、梯度范数、长期闲置率、输出相似度和召回重叠。若有容量限制，还需检查实际执行负载，而不只是容量截断前的 routing。

迁移边界： MoE 的 expert 是全局共享计算模块；多兴趣模型的输出通常是每个用户生成的向量，槽位编号不一定有跨用户固定语义。借鉴负载均衡是设计选择，不是两个对象完全等价。

12. Switch 均衡损失：两个比例为什么相乘

12.1 我的问题：“频率直接 MLE 不就行了？”

若只是估计类别使用频率，确实可以：

$$f_k = \frac{n_k}{B}.$$

例如 100 个输入分到三个 expert 的数量是 90、8、2，则比例为 0.90、0.08、0.02。

但这是测量当前负载，不会自动产生优化方向。hard argmax 的计数对 router 参数几乎处处不可导，还需要可微代理或离散分配算法。

12.2 f 和平均概率究竟差在哪?

为避免把 top-1 与 top-2 的归一化混用，先只讲 top-1。令 $p_k(x)$ 是 top-1 之前的 softmax 概率：

$$f_k = \frac{1}{B}\sum_x \mathbf{1}[\arg\max_j p_j(x) = k], \qquad P_k = \frac{1}{B}\sum_x p_k(x).$$

100 条输入每条都给 A/B 概率 0.51/0.49，则：

$$f = (1, 0), \qquad P = (0.51, 0.49).$$

原因是 top-1 选最大值，不是按概率抽签。一个是实际决策频率，一个是决策前的平均软概率。

若每条有多个路由，f_k 的分母和 gate 是否重归一化必须另行定义；不要把 top-1 公式未经说明搬过去。

12.3 一个最直观的均匀分配正则

我们可以先构造：

$$L_{\text{simple}} = \sum_k (P_k - 1/K)^2.$$

它鼓励 batch 平均概率接近均匀，但平均概率接近均匀不保证 hard top-1 使用均衡。上面的 0.51/0.49 就说明了这一点。

这不是每个输入都要均匀；不同输入可以做尖锐而不同的选择，最后平均起来均衡。

12.4 原始 Switch 风格的代理目标

Switch 使用如下辅助项：

$$L_{\text{bal}} = \lambda K \sum_k f_k P_k.$$

训练时通过 P 回传，f 视为停止梯度的实际选择统计。它关注的是“当前拥挤的 expert 是否还被分到大量软概率”。

我的追问： 为什么不 cosine 或 triplet？直接乘起来有什么意义？

cosine 若只鼓励两个分布相似，两个分布都集中在同一个 expert 时也可能很好，却没有均衡。triplet 本来表达正负对象的相对距离，这里没有自然对应的正负三元组。

点积在这里可看成一个成本：实际负载大的 expert 获得更高的概率分配价格。

12.5 把梯度真正写出来

对平均概率：

$$\frac{\partial L_{\text{bal}}}{\partial P_k} = \lambda K f_k.$$

我的推导： f 大，就减去更大的梯度，所以降低概率。

方向直觉合理，但实际参数是 softmax 前的 logits，不是能独立修改的 P。若单输入 logit 为 a_{xj}，则：

$$\frac{\partial L_{\text{bal}}}{\partial a_{xj}} = \frac{\lambda K}{B} p_j(x) \left[f_j - \sum_k f_k p_k(x) \right].$$

这说明：负载高于该输入概率加权平均的 expert 受到向下压力，低于平均的受到向上压力。f_j 大不意味着每个条件下都能孤立地直接减一个同样大小的 logit。

12.6 它不是严格均衡保证

如果所有 f 都相等，在该步把 f 固定，$\sum_k f_k P_k$ 对所有合法 P 都相同，router 的这个辅助梯度为零。

因此不能把它描述为“对两个自由分布做严格距离最小化”，也不要声称只要加它就保证所有 expert 平均更新。

噪声路由、均衡正则、容量控制、带配额分配等都是工具，但作用不同。单纯设置容量上限也可能只是丢弃溢出 token，并没有让闲置 expert 自动得到训练。

13. MIND 原论文实际做了什么

讨论到这里，我要求对照原始论文，而不是把我们发明的组合误记成原版。

原版关键点： MIND 用行为到兴趣的 dynamic routing 生成多个兴趣；路由 logits 随机高斯初始化以打破相同兴趣的对称；训练时用 label-aware attention 选取与目标物品相关的表示，再采用 sampled softmax。它没有刚才 Switch 风格的均衡 loss。论文实验中，label-aware attention 趋向 hard selection 时效果与收敛更好。

这里有两种不同的路由，不要混淆：

1. 历史行为 → 兴趣中心：生成兴趣时的软聚类。
2. 兴趣中心 → 目标监督：训练时选择相关兴趣。

为了理解第一种，可用如下**概念性迭代**，省略原文 squash 和完整结构：

$$w_{ik} = \text{softmax}_k(b_{ik}), \qquad c_k = \text{Aggregate}_i(w_{ik} T e_i),$$

$$b_{ik} \leftarrow b_{ik} + c_k^\top T e_i.$$

这是“按匹配分配，再根据聚合结果增强一致性”的前向过程。外层推荐 loss 训练 T、embeddings 和后续网络参数，不能把 b 的前向更新说成独立 optimizer step。

线上生成多个候选无关用户向量，各自 ANN 检索，再合并候选。训练时 label-aware 选择可以用目标物品，但用户兴趣生成不能因此偷看未来目标。

讨论修正： “温度变软更好”“加均衡一定更好”都不是原论文结论；随机初始化这个建议倒与原文有直接对应。

14. 海量数据能否自然解决兴趣闲置

我的反驳是：如果有十亿、百亿甚至更多 session，K 只有一百，应该都有机会更新，不一定值得担心。

这个判断有合理部分：更多且多样的数据增加不同分支被使用的机会。也不能假定某个大模型必然塌缩。

但数量不提供保证。如果路由长期偏好同一个分支，更多数据也可能加强它。决定机会的是数据、初始化、共享参数和路由动态，而不只是 B/K。

多兴趣槽位还常有排列对称性：用户 A 的第一个槽不一定与用户 B 的第一个槽代表同一主题。slot 使用统计可以发现某些优化问题，但不能直接当作跨用户固定语义分工。

因此应先建立 baseline 并监控，而不是预先强求使用分布越均匀越好。真实偏好有主次，有些用户只需要少量有效兴趣；为了填满 K 个槽而拆碎一种兴趣，未必改善召回。

15. 用户模型总表与时间线

方法	生成什么	主要改善的表达问题
User ID lookup	一个可学习用户向量	记住长期偏好，但对新行为响应有限
历史 pooling	平均或动作/时间加权总结	直接用已知行为，而不只依赖 ID
GRU/LSTM encoder	顺序递推状态	表达顺序与选择性记忆
Self-attention / SASRec	前缀上下文状态	更直接访问不同历史位置
DIN	候选相关历史聚合	对不同候选提取不同相关历史
DIEN	兴趣序列与候选相关演化	加中间监督，刻画变化过程
多兴趣 / MIND	K 个用户兴趣向量	减少单向量对多个偏好的压缩

时间线不是性能排行榜：

模型	时间口径	作者/团队
GRU	2014	Cho 等
GRU4Rec	2015 预印本 / 2016	Hidasi 等
DIN	2017 预印本 / KDD 2018	阿里
SASRec	ICDM 2018	Kang、McAuley
DIEN	2018 预印本 / AAAI 2019	阿里
MIND	CIKM 2019	阿里及合作研究者

SASRec 与 DIEN 是当时并行的路线，不要画成“DIEN 失败后才发明 attention”。架构可以借鉴组合，也不意味着一个完全替代另一个。

Part 1 的另一条路线则是 FM（2010）、Wide & Deep（2016）、DeepFM/DCN（2017）、DLRM（2019）、DCNv2（2020 预印本 / WWW 2021）。它们主要讨论特征交互，不是完全相同的历史编码问题。

16. 统一实验能说明什么，不能说明什么

我要求有整体数据比较，而不是每篇论文都说自己的方法胜出。

Tenrec 是一个有用入口。以下是作者公开的特定子集/协议结果，不代表全量工业通用排序。

CTR-1M shared embedding	AUC
NFM	0.7924
DCNv2	0.7922
DeepFM	0.7920
DIEN	0.7918
DCN	0.7911
DIN	0.7910
Wide & Deep	0.7910

Session recommendation 1M	NDCG@20
SASRec	0.0194
GRU4Rec	0.0192
BERT4Rec	0.0185

这里不能把两张表横比，也不能仅凭点估计宣称显著差异。共享 embedding 的具体实验设置尤其不该与 separate embedding 的另一张榜混用。

MIND 原论文 Tmall 的 HitRate@100：YouTube DNN 为 0.1648，MIND 单兴趣为 0.1930，MIND 五兴趣为 0.2699。这支持其特定实验中的多兴趣价值，不支持所有数据上都同等提升。

2024 年序列推荐复现实验还指出：SASRec 并非所有设置都胜过 GRU4Rec，模型规模与实验协议会影响结论。结构、loss、采样和调参都必须拆开。

17. Loss 的分界：训练单位，不是输出有没有标量

17.1 我的统一反传理解

我的观点： 最后每个 document 都有一个分数。不同 loss 只要提供每个分数的导数，就能通过同一个 f 回传。

对。若 $s_i = f_\theta(q, d_i)$，有：

$$\frac{\partial L}{\partial \theta} = \sum_i \frac{\partial L}{\partial s_i} \frac{\partial s_i}{\partial \theta}.$$

但“每个结果一个分数”不等于 pointwise 训练。pointwise、pairwise、listwise 的区别，是 loss 依赖单条、相对 pair，还是整组列表。

本文假设共享、逐候选打分 f；若 scorer 本身用列表上下文，输出分数间还会有结构耦合，但对完整计算图依旧按链式法则反传。

17.2 三类目标的核心对比

类别	样本/监督组织	关注什么
Pointwise	单个 q-document 及 label	单项点击概率或相关性
Pairwise	同一 q 下两个结果及偏好	谁应该排在谁前面
Listwise	同一 q 的整组候选	整体相对概率或排列

BCE 学的是观察协议下的点击概率。若位置偏差、负采样或延迟反馈改变监督，概率意义也会变；不是一个 sigmoid 自动带来真实偏好概率。

18. Pairwise logistic：从三元组到两边的梯度

18.1 batch 究竟是什么

我的疑问： 是否要在一个 batch 的所有结果之间，两两计算 delta？query 放在哪里？

最直接是一条训练记录：

$$(q, d^+, d^-),$$

同一 q 下，正例/更相关结果应高于负例/较低相关结果。batch 可含 B 个这样的三元组。另一个实现是 batch 中放多个 query 列表，再在每个列表内部构造有效 pair。

不必枚举所有 pair，也通常不跨无关 query 比较。一个结果可以参加多个 pair，统计与权重仍要考虑相关性。

18.2 从“胜出概率”到 CE

$$s^+ = f_\theta(q, d^+), \quad s^- = f_\theta(q, d^-), \quad \Delta = s^+ - s^-.$$

把胜出概率定义为：

$$p_{+>-} = \sigma(\Delta).$$

偏好 label 为 1，则：

$$\ell = -\log \sigma(\Delta) = \log(1 + e^{-\Delta}).$$

这就是常见 pairwise logistic 形式。实践中可用 softplus$(-\Delta)$ 保持数值稳定。

对于软偏好 $y \in [0, 1]$，可使用 BCEWithLogits(Δ, y)。不要把所有 noisy clicks 自动当成确定偏好。

18.3 “再传回两个打分分支就行了”

确实如此：

$$\frac{\partial \ell}{\partial \Delta} = \sigma(\Delta) - 1,$$

$$\frac{\partial \ell}{\partial s^+} = \sigma(\Delta) - 1, \qquad \frac{\partial \ell}{\partial s^-} = 1 - \sigma(\Delta).$$

正例导数为负，梯度下降倾向提高其分数；负例导数为正，倾向降低其分数。

例如 $\Delta = 0$，两边分别为 -0.5、+0.5。若正例已明显胜出，梯度会减小。若排序反了，修正压力更大。

它们通常共享同一个打分模型，参数梯度累加：

$$\nabla_\theta \ell = (\sigma(\Delta) - 1)\left(\nabla_\theta s^+ - \nabla_\theta s^-\right).$$

不是两个互不相干的 pointwise 模型，也不是必须手写反向传播。

18.4 一条重要不变性

给同一 query 的全部分数加常数 c：$(s^+ + c) - (s^- + c) = \Delta$。pairwise logistic 不会因此变化。

所以它不要求每条结果的绝对点击概率准确。排序目标和校准目标是两件事，必要时可以联合训练，但应承认目标之间可能存在取舍。

19. Hinge、BPR 与 NDCG 加权梯度

19.1 Hinge / margin ranking

$$\ell_{\text{hinge}} = \max(0, m - (s^+ - s^-)).$$

达到 margin m 后，该 pair 的梯度为零；logistic 则平滑衰减，并不在有限分差处严格停止。

hinge 不是 L1 正则。一个惩罚排序间隔不足，另一个通常惩罚参数大小。

19.2 RankNet 与 BPR 的公式相似，不等于完整方法相同

RankNet 的 pairwise 概率建模和 BPR 常见的正负偏好项，都可以出现 $-\log \sigma(s^+ - s^-)$。但动机、训练 pair 的形成、采样、先验/正则和应用背景不同。

不应只因 loss 局部形式一样，就说整篇方法一样。尤其 BPR 中未观察物品是相对偏好构造，不是日志证实的真实不喜欢。

19.3 “listwise 都是 NDCG 模拟梯度？”

这更接近 LambdaRank 的直觉，而不是全部 listwise 目标。

对同一 query 的两个位置 i、j，交换产生的 $|\Delta \text{NDCG}_{ij}|$ 可用于加权 pairwise 梯度，让影响排序指标更大的错序得到更强压力。

教学形式可写：

$$\tilde{\ell}_{ij} = w_{ij} \log(1 + e^{-(s_i - s_j)}), \qquad w_{ij} = |\Delta \text{NDCG}_{ij}|.$$

每次计算当前排名与 w 时通常不对排序操作求导，将 w 视作该步权重。这个写法说明梯度构造，不意味着原始离散 NDCG 已经变成普通光滑可微函数。

LambdaMART 将 lambda 思路用于提升树；同样的梯度思想也能用于神经 scorer。不要把它和“所有 listwise 都必须这样”画等号。

20. Listwise：列表概率，而不只是模拟 NDCG 梯度

本节是对前面口头讨论的**补充对齐**：我们尚未逐一独立推完所有 listwise 原论文，下面给出最小公式来避免概念混淆。

20.1 ListNet 风格 top-one 分布

对一个 query 的 n 个候选，模型分布：

$$p_i = \frac{e^{s_i}}{\sum_j e^{s_j}}.$$

用相关性标签经某个明确变换 g 构造目标分布：

$$q_i = \frac{e^{g(y_i)}}{\sum_j e^{g(y_j)}}, \qquad L = -\sum_i q_i \log p_i.$$

梯度非常直接：

$$\frac{\partial L}{\partial s_i} = p_i - q_i.$$

每个结果仍收到一个标量梯度，但 p 的分母依赖整组候选。这就是 listwise 耦合，不需要模拟 NDCG 梯度。

20.2 它与 in-batch CE 为什么看起来相似

若 q 只在一个正例上为 1，其余为 0，就变成在候选集合中选择正例的 softmax CE。

数学形式相似，训练语义却可能不同：检索 in-batch 候选往往来自采样，query 排序列表往往来自检索或曝光。负例分布、去重、logQ 和列表资格不同，不能看公式相似就忽略数据构造。

20.3 ListMLE 的排列似然

若目标排列为 π，一个经典形式为：

$$L = -\sum_{t=1}^{n} \log \frac{e^{s_{\pi_t}}}{\sum_{j=t}^{n} e^{s_{\pi_j}}}.$$

直觉是：先从全列表选第一名，再从剩余候选选第二名，依次计算目标排列的概率。存在同级标签时，需要定义 tie 的处理，不能随便把任意顺序当确定偏好。

它也能正常求导，不需要先定义一个 NDCG surrogate gradient。

20.4 四类目标收拢

目标	loss 根据什么变化	分数梯度来自哪里
Pointwise BCE	单项预测与 label	$\sigma(s_i) - y_i$
Pairwise logistic	正负分差	对 pair 的两边施加相反导数
ListNet 风格 CE	列表预测分布与目标分布	$p_i - q_i$
LambdaRank 思路	pair 错序与指标影响	指标变化加权的 pairwise lambda

没有一种可以脱离数据与业务目标说“永远最好”。

21. 模型、数据、loss、优化的实验怎么拆

为弄清提升究竟来自哪里，可以分阶段对比，而不是一次换全部组件。

实验	固定什么	改什么	回答什么
历史 encoder	特征截止、目标、候选、预算	pooling / GRU / attention	序列建模是否提供增益
多兴趣容量	encoder 家族、候选、loss	K=1 与 K>1，报告容量	多个表示是否值得成本
辅助监督	主结构、主任务	aux 开/关、权重	中间监督有无增益
路由	encoder、数据、容量	hard / soft / balancing	分工和训练稳定性如何变化
排序目标	scorer、切分、label、采样	BCE / pairwise / listwise	学习目标改变是否更符合指标

每组最好报告效果大小、评估不确定性、训练 seed 波动、训练成本、推理延迟及缓存/索引要求。

数据预处理、序列长度、负采样、全量与采样评估、模型大小、训练 token/step 预算都应说明。只写“AUC 提升”而不说明这些，无法归因。

正则的作用也要观察两个结果：是否改善使用率或向量差异，以及是否改善主任务。辅助指标好看不等于推荐更好。

22. 面试与 ML coding：把推演落到一个能跑的闭环

我最终的判断是：面试讨论中，需要能解释建模动机与数据/目标；ML coding 不必每个论文都逐层从零复现，可以以之前的双塔 mock 为基础练一个闭环。

22.1 能明确写出的形状

history_ids [B, T]
history_embeddings [B, T, d]
history_mask [B, T]
user_state [B, d]
multi_interest_states [B, K, d]
candidate_embeddings [B, N, d]
scores [B, N]
pair_scores [B, 2]

B、T、K、N、d 各是什么意思，要在一开始定义。相似公式的实现错误，常常来自轴混淆，而不是不懂论文。

22.2 一个 pairwise loss 的最小实现

```
import torch
import torch.nn.functional as F

def pairwise_logistic_loss(pos_score, neg_score, weight=None):
  # pos_score / neg_score: [B]
  loss = F.softplus(neg_score - pos_score)
  if weight is None:
    return loss.mean()
  # Require finite, nonnegative weights and a positive weight sum.
  return (weight * loss).sum() / weight.sum()
```

这是计算形状示意，不是完整训练项目，本文没有把它作为端到端程序执行。面试中还需要根据输入契约检查空集合、全 padding、零权重等边界。

22.3 真正要熟练的检查

- 正例与负例有没有配反，是否属于同一 query。
- mask 是否排除了 padding，而不是误当负例。
- label 有没有进入预测输入。
- 梯度是否到达 embeddings、history encoder、共享 scorer。
- pairwise 导数方向是否合理，小数据能否过拟合。
- eval 模式与关闭 autograd 是否分别正确设置。
- 召回时用户向量是否候选无关，索引是否版本兼容。

“核心思想推完”不等于“原版全部细节和实现已经掌握”。把四个接口对齐：输入、关键计算、输出、loss，再写小规模验证，比继续堆模型名更实用。

23. 纠错记录、接续问题与参考文献

23.1 这次往返中真正值得保留的修正

初始说法	更准确的理解
GRU 把上一个输入和当前输入结合	结合的是旧隐藏状态与当前输入
门控像 ReLU 阈值激活	比例门通常用 sigmoid，是软开关
状态接近 ±1 就重要	激活幅度不是语义重要性的定义
Attention 明显就是语义	它明确计算匹配，但语义仍由监督形成
给门加 loss 就保证记住正确内容	辅助任务提供压力，不保证内部解释
K 个向量就是一个 K 维输出	是 K×d，中间激活由上游网络生成
未选中兴趣都应远离正例	未选中不等于负例
L2 normalization 能防闲置	它只处理尺度，闲置需要单独检查
soft label 解决塌缩	这里叫软路由，只改善梯度路径
f 和 P 都是使用率	一个是 hard 频率，一个是平均软概率
Switch 均衡就是 MIND 原版	是我们借鉴的扩展，原版没有该项
海量数据必然让所有兴趣更新	机会取决于路由与优化动态，不能保证
标量打分就是 pointwise	分类取决于 loss 的训练单位与耦合
Listwise 都是 NDCG 模拟梯度	ListNet/ListMLE 可直接对概率目标求导

23.2 本篇刻意没有假装已经完成的内容

完整复现 AUGRU、MIND 的全部前向与反传细节、不同 LambdaLoss 的理论性质，以及各种工业路由算法，都没有在这次口头推演中逐项完成。

后续若需要，应优先挑一个结构和一个目标，固定数据协议跑可解释的小实验。模型与 loss 的核心地图已经足够支撑这种深入，不必先背所有衍生名称。

23.3 参考资料

1. Cho et al. **Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation.** 2014. Paper。用于核对 GRU 思路；本文统一使用“z 为写入比例”的符号。
2. Bahdanau et al. **Neural Machine Translation by Jointly Learning to Align and Translate.** ICLR 2015. Paper。固定长度编码瓶颈与按需访问历史的背景。
3. Hidasi et al. **Session-based Recommendations with Recurrent Neural Networks.** Paper。GRU4Rec 时间线。
4. Zhou et al. **Deep Interest Network for Click-Through Rate Prediction.** KDD 2018. Paper。Part 1 已推的候选相关聚合。
5. Zhou et al. **Deep Interest Evolution Network for Click-Through Rate Prediction.** AAAI 2019. Paper。中间辅助监督及兴趣演化。
6. Kang and McAuley. **Self-Attentive Sequential Recommendation.** ICDM 2018. Paper。结构与原始 BCE 训练目标应分别对齐。
7. Li et al. **Multi-Interest Network with Dynamic Routing for Recommendation at Tmall.** CIKM 2019. Paper。dynamic routing、随机初始化与 label-aware selection。

8. Fedus et al. **Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity.** JMLR 2022. Paper。top-1 负载均衡代理，不是 MIND 原版的一部分。
9. Burges et al. **Learning to Rank using Gradient Descent.** ICML 2005. Official page。RankNet。
10. Rendle et al. **BPR: Bayesian Personalized Ranking from Implicit Feedback.** UAI 2009. Paper。注意 arXiv 上传年份与发表年份不同。
11. Cao et al. **Learning to Rank: From Pairwise Approach to Listwise Approach.** ICML 2007. Official page。ListNet。
12. Xia et al. **Listwise Approach to Learning to Rank: Theory and Algorithm.** ICML 2008. DOI。ListMLE。
13. Burges. **From RankNet to LambdaRank to LambdaMART: An Overview.** 2010. Official page。指标加权梯度的背景。
14. Yuan et al. **Tenrec: A Large-Scale Multipurpose Benchmark Dataset for Recommender Systems.** NeurIPS 2022. Project and result tables。
15. Betello et al. **A Reproducible Analysis of Sequential Recommender Systems.** 2024. Paper。统一设置与性能排序变化。

参考文献用于核对具体方法。正文的反驳、梯度展开、实验建议和教学变体是本次讨论的整理，不应被误读为每篇论文的全部贡献或原始推导顺序。